Technical Assignment: Large-Scale Excel Import

Role: Senior PHP/Laravel & Angular Developer | **Stack:** Laravel REST APIs, Angular, MySQL/PostgreSQL | **Suggested time:** 3–5 working days

1 Objective

Build a web application to upload, validate and import an **.xlsx file containing up to 500,000 records per upload**. The solution must use asynchronous processing, scale without excessive memory usage, and remain reliable when records or jobs fail.

2 Excel Fields & Validation

Field	Validation	Field	Validation
Customer ID	Required; unique	**First Name**	Required
Last Name	Required	**Email**	Required; valid email
Phone Number	Required; valid format	**Date of Birth (DOB)**	Required; valid date
Gender	Optional	**Profession**	Required
Company Name	Optional	**Country**	Required
City	Required	**Address**	Optional
Social Media URL	Optional; valid URL	**Registration Date**	Required; valid date
Status	Active / Inactive		

3 Core Functional Requirements

- **Upload & processing:** Validate file type, size and structure; securely store the upload; return an import ID quickly; process records in configurable chunks using Laravel queues/jobs and efficient bulk database operations. Do not process the full import in the HTTP request.
- **Record validation:** Validate each row on the backend. Continue with valid rows even when some records are invalid. Capture the Excel row number and reason for every rejected row.
- **Duplicate prevention:** Detect duplicate Customer IDs within the file and already in the database. Prevent duplicates under concurrent uploads and job retries using appropriate database constraints and idempotent logic.
- **Failure recovery:** Handle interrupted uploads, corrupt files, database errors, worker crashes and partial imports. Use logging, controlled retries and safe restart/resume behavior without re-inserting committed records.

4 Angular Progress Popup (Required)

Show a clear, user-friendly **popup/modal** with three distinct progress indicators that update automatically from real backend status:

01	**File upload** Percentage of file transferred to server.
02	**Data validation** Rows checked; valid, invalid and duplicate counts.
03	**Database import** Validated rows processed and successfully saved.

Display current stage, total/processed/successful/invalid/duplicate counts, percentages, final result and errors. Users must be able to close or minimize and reopen the popup **without stopping processing**. Upload progress and backend processing progress must be clearly distinguished.

5 Import History & Error Reporting

Provide an import history screen with filename, status, timestamps, processing duration and record counts. Show paginated row-level validation errors and allow download of a CSV error report (row number + reason). Include Queued, Processing, Completed, Completed with Errors and Failed statuses.

6 Engineering Expectations

- Follow clean Laravel architecture (controllers, services, jobs, validation, migrations), consistent REST APIs, proper status codes, transactions, indexes and unique constraints.
- Use secure file storage, server-side checks, authenticated/authorized access to imports and downloads, safe error messages, and protection against unsafe spreadsheet content.
- Build reusable Angular components/services with TypeScript types, responsive UI, loading/error states and efficient polling or other suitable progress updates.

7 Testing & Performance

Test with a **500,000-record Excel file**, including invalid/duplicate rows, concurrent imports and a failed-job retry. Provide automated tests for critical behavior, plus processing time, peak memory usage, chunk size and queue/database configuration. There is no fixed time benchmark; explain performance choices.

8 Deliverables

Share a **Git repository** with Laravel + Angular source code, migrations, setup README, brief architecture notes, API documentation (Postman/OpenAPI), automated tests and 500,000-row performance results. Be prepared to demonstrate failure recovery and explain key design decisions.

Evaluation focus: scalable architecture, correct data validation and duplicate handling, resilience to failures, clear Angular progress UX, code quality and measured performance.